Orden de crecimiento asintótico

Santiago Javier Proaño Yépez

<2026-10-13 mar>

Contents

1 Introducción **1**

2 Notación Big-O (Big-O) **1**
2.1 Explicación ATTACH 2

3 Notación Big- (Big-omega) **3**
3.1 Explicación ATTACH 4

4 Notación Big- (Big-theta) **5**
4.1 Explicación ATTACH 5

1 Introducción

El tiempo de ejecución de un algoritmo crece en proporción a su tamaño de entrada n, y como depende de cada computadora, a veces n se tiene como constante. Pero al analizar los algoritmos se deben ignorar estas constantes mediante otras funciones, mismas que devuelven solo la parte pesada de los algoritmos, es decir lo que al aumentar n más genera peso en el tiempo de ejecución. Es aquí donde entra el análisis asintótico.

Este análisis se lo hace mediante 3 funciones que en transforman todo el conteo de pasos de T(n) Análisis de algoritmos y notación asintótica en tiempo de ejecución.

2 Notación Big-O (Big-O)

Sea f(n) una función para medir los pasos de un algoritmo y g(n) una función simple para comparar, se dice que

$$f(n) \text{ es } O(g(n))$$

o es Big-O de g(n) si existe una constante $C > 0$ y una constante entera $n_0 \geq 1$ de modo que

$$f(n) \leq Cg(n)$$

para $n \geq n_0$

2.1 Explicación ATTACH

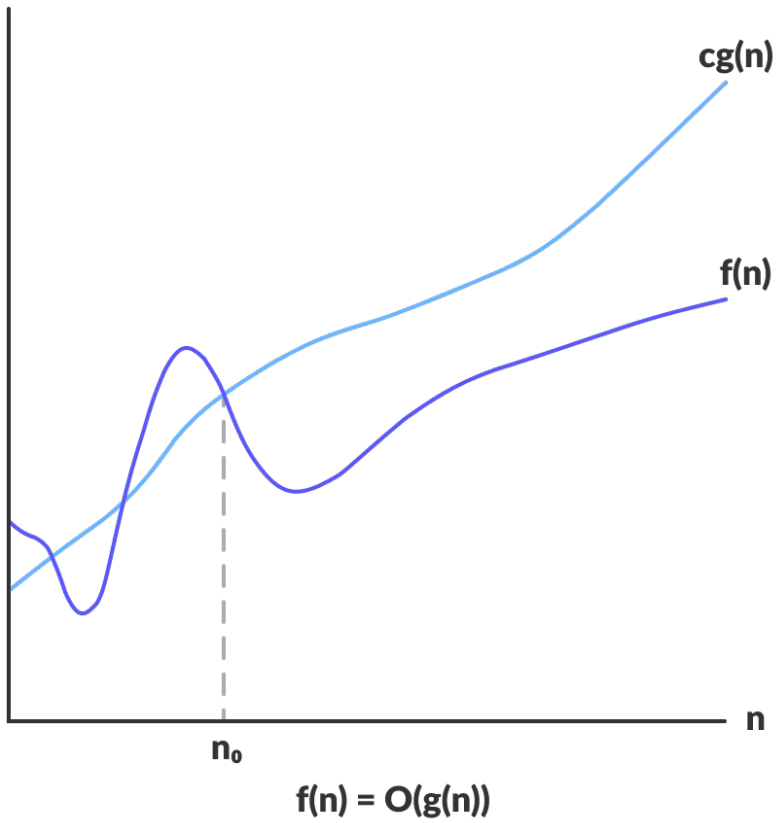
Digamos que tenemos a la función $T(n) = f(n) = 8n - 2$ entonces se dice que este 8n - 2 es Big-O de n, (n seria g(n) por que es una función simple para comparar) si encontramos una constante entera a partir de 1, desde donde crecerá n, y una constante C positiva que al multiplicar por esta g(n) y reemplazar n a partir de esta constante será mayor o igual que la f(n).

En el ejemplo del libro tenemos que $C = 8$ y $n_0 = 1$, entonces $n \geq 1$ como lo verificamos? reemplazamos

$$\begin{aligned} 8n - 2 &\leq Cn \\ n = 1, C &= 8 \\ 6 &\leq 8 \end{aligned}$$

Y así podemos ir subiendo n y verificamos que en efecto f(n) siempre sera menor o igual que esa Cg(n) ya que si restamos el 8n por el dos siempre sera menor o igual que solo tener 8n. Por lo tanto f(n) es O(n). Ojo estas constantes no son únicas pueden haber varias.

Es decir, 8n - 2 no crecerá más rápido que una 8n. Entonces Big-O determina un techo de crecimiento de una función, de modo que esta función crecerá a un ritmo similar que su techo pero no tendrá un ritmo de crecimiento más rápido. Además siempre buscará que ese techo sea el más ajustado para f(n).



Si te das cuenta en la grafica a partir de ese n0 es donde el crecimiento de f(n) es similar que su techo Cg(n)

Es profundiza en Notación Big-O

3 Notación Big- (Big-omega)

Sea f(n) una función de los pasos de un algoritmo y g(n) una función simple para comparar se dice que

$$f(n) \text{ es } \Omega(g(n))$$

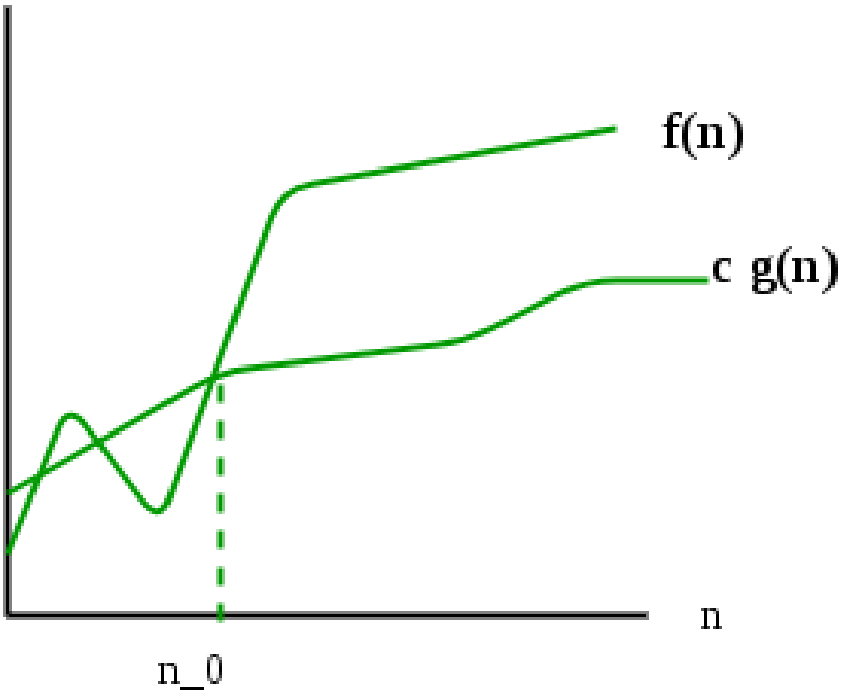
Si hay una constante $C > 0$ y una constante entera $n_0 \geq 1$ tal que:

$$f(n) \geq Cg(n)$$

para $n \geq n_0$

3.1 Explicación ATTACH

Es lo contrario al Big-O, en este caso se busca una constante positiva real, que al multiplicar por un g(n) y una constante n0 desde donde crecerá n, crezca g(n) a un ritmo similar al de f(n) pero nunca será mayor. O sea se busca un suelo desde donde f(n) crece a un ritmo similar pero siempre será mayor o igual que ese suelo. Al igual que en big-O se debe ajustar ese suelo lo más posible a f(n).



f(n) = Omega(g(n))

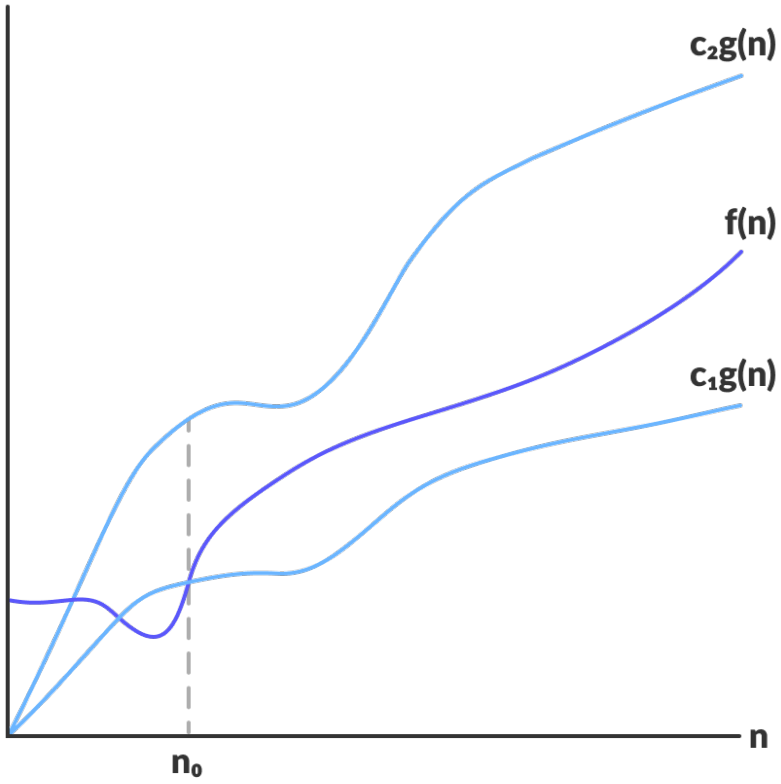
4 Notación Big- (Big-theta)

Permite decir que dos funciones crecen al mismo ritmo hasta factores constantes. Entendiendo las dos grandes notaciones anteriores se dice que f(n) es (g(n)) si es O(g(n)) y (g(n)). Entonces hay constantes $C_1 > 0$ y $C_2 > 0$ y una constante entera $n_0 \geq 1$ tal que:

$$C_1 g(n) \leq f(n) \leq C_2 g(n)$$

4.1 Explicación ATTACH

Siguiendo la lógica de las anteriores notaciones en este caso el big theta busca un suelo y un techo por el cual una función f(n) crecerá, a un ritmo no menor que su suelo, ni mayor que su techo. Acota el crecimiento de f(n) entre dos constantes.



$f(n) = \Theta(g(n))$